Engineering Brief: EU Job Market Scraper & Dashboard

Issued by: Bolders CG
Status: Open for development
Time estimate: 2–3 days

What We're Building

A data pipeline that harvests European job market data across all employment types — permanent, contract, and freelance — normalises it into a clean, filterable schema, and surfaces it through a dashboard. The dataset must be structured for a prediction model to be layered on top later.

Problem

We need to understand demand patterns for roles across Europe — by country, industry, job function, seniority, rate, employment type, and language requirement. No single source gives us this. We need to aggregate, normalise, and make it explorable.

Data Sources

You are responsible for finding the most reliable, scrapable version of each. These are starting points — improve on them.

Source	What we want
LinkedIn Jobs (NL / DE / FR)	All employment types, rich on industry, seniority, company size
Indeed NL / DE / FR	Broad coverage, salary ranges, all employment types

Malt.com	Freelance/contract rates, skills, language, country
Freelancer.eu or equivalent	Freelance project budgets, role categories, country
EURES (EU Jobs portal)	Official EU cross-border job data, permanent and contract
Kaggle	At least one public dataset relevant to job market trends, salary benchmarks, or LinkedIn postings — your call, justify your choice in the README

Scrape broadly. Do not pre-filter by employment type, work arrangement, or industry. Capture everything — we filter in the dashboard, not the scraper.

Data Schema (minimum, must be extensible)

```
id
source
role_title
role_category
job_function              # data / dev / design / marketing / ops / finance / oth
industry                  # tech / pharma / finance / retail / logistics / other
seniority_level           # junior / mid / senior / lead / director / unknown
company_size              # startup / smb / enterprise / unknown
employment_type           # permanent / contract / freelance
work_type                 # remote / hybrid / onsite
country
language_required         # explicit in posting or null
rate_raw
rate_normalized_eur_day
rate_type                 # hourly / daily / monthly / annual / project
skills[]
posted_date
scraped_date
```

Inference rules: Where fields are not stated, infer from context (job title → seniority, company name → industry via lookup). Document confidence logic. Null is better than a wrong value — flag low-confidence inferences.

Rate normalisation logic is your call — document your assumptions.

Dashboard

Filters on every schema field. Charts showing:

- Demand by country
- Demand by industry and job function
- Rate ranges by role category and seniority
- Employment type breakdown (permanent / contract / freelance)
- Work type breakdown (remote / hybrid / onsite)
- % of postings with explicit language requirement
- Date range filter across all views
- Export current filtered view to CSV

Stack: your choice. Justify it briefly in the README.

Repo & Deployment

GitHub

- Private repo, clean structure
- README: setup instructions, data source rationale, schema docs, inference logic
- `.env.example` committed, no secrets ever committed
- Conventional commits

Local (dev only — prod is a later step)

```
docker compose up
```

That's it. Must work first time.

Open Ends — Figure These Out

These are not specified intentionally. Your judgement is being tested.

- How to handle rate normalisation across hourly / daily / monthly / annual / project-based

- How to scrape without getting blocked across 5+ sources
- Which Kaggle dataset adds the most signal for future prediction
- How to parse multilingual postings (NL, DE, FR) into a consistent schema
- How to infer industry and seniority reliably where not stated
- How often to refresh — propose a schedule and defend it

Done When

- Dashboard loads with real data from at least 4 sources
- All schema fields present and documented
- Filters work across all dimensions
- CSV export works on filtered view
- Runs locally with one command
- Deployed and live on Hetzner
- README explains every non-obvious decision

What Comes Next (not your scope now)

A forecasting layer will be built on top of this dataset — demand prediction by industry × role × country × employment type. Design the schema with time-series modelling in mind. If you make decisions that would break that later, flag them now.

Questions? Raise them before starting, not halfway through. This should be fun — go explore.